Session Types : Lemmas and Proofs

Tadayoshi Kamegai

Contents

1 Introduction **1**

2 Definitions **2**
2.1 Process Calculus 2
2.1.1 Designing Processes in Coq 4
2.2 Local Types and Trees 4
2.3 The Merging Operator 5

3 Theorems **6**
3.1 The Subject Reduction Proof 6
3.2 From a Coq Implementation Perspective 8

4 Balanced Graphs and Trees **9**
4.1 Liveness 10

5 Representations (loosely) **11**
5.1 On Trees 11
5.2 Liveness in Rocq 12

1 Introduction

This note aims to provide a complete proof chart for basic / important results regarding session types with coq implementation in mind.

2 Definitions

Notation 2.0.1. We give the following base sets:

- values : $\mathrm{v}, \mathrm{v}', \ldots$
- expressions : $\mathrm{e}, \mathrm{e}', \ldots$
- expression variables : $x, y, z, \ldots$
- labels : ℓ, ℓ'
- session participants : $p, q, \ldots$
- processes : $P, Q, \ldots$
- process variables : $X, Y, \ldots$
- sessions : $\mathcal{M}, \mathcal{M}', \ldots$

Values are ranged over by , expressions ranged over by , expression variables ranged by , labels

2.1 Process Calculus

Processes are defined by

$$P ::= p!\ell\langle \mathrm{e} \rangle.P \mid \sum_{i \in I} p?\ell_i(x_i).P_i \mid \text{if e then } P \text{ else } P \mid \mu X.P \mid X \mid \mathbf{0}$$

The output process $p!\ell\langle \mathrm{e} \rangle.P$ is one which sends the value of expression e with label ℓ to participant p. The input process $\sum_{i \in I} p?\ell_i(x_i).P_i$ is a process that can accept a value with label ℓ_i from participant p for any $i \in I$. The variable x_i is instantiated with the value received in the continuation process P_i. In general, I is finite and non-empty.

The conditional process represents the internal choice if e then P else Q between P and Q. $\mu X.P$ is a recursive process.

In order to make some sense about what the process represents, we define an unrolling function.

Definition 2.1.1 (Unrolling of Processes)**.**

We also assume that recursive processes are guarded with input and output processes. For instance, $\mu X.p!\ell(x).X$ is a valid process, but $\mu X.X$ is not. We can define this condition inductively by keeping track of which variables are guarded. That is,

$$\frac{}{\mathrm{guard}_P\ \{X\}\ X} \qquad \frac{\mathrm{guard}_P\ \emptyset\ (p!\ell\langle \mathrm{e} \rangle.P)}{\mathrm{guard}_P\ A\ P} \qquad \frac{\mathrm{guard}_P\ \emptyset\ (\sum_{i \in I} p?\ell_i(x_i).P_i)}{\mathrm{guard}_P\ A_i\ P_i \quad i \in I}$$

$$\frac{}{\mathrm{guard}_P\ \emptyset\ \mathbf{0}} \qquad \frac{\mathrm{guard}_P\ (A \cup B)\ (\text{if e then } P \text{ else } Q)}{\mathrm{guard}_P\ A\ P \quad \mathrm{guard}_P\ B\ Q} \qquad \frac{\mathrm{guard}_P\ A\ (\mu X.P)}{\mathrm{guard}_P\ A\ P \quad X \notin A}$$

We then say that a process P is **guarded** (and closed) if we can infer that $\mathrm{guard}_P\ \emptyset\ P$.

By keeping track of which variables have been used, we can also ensure that things like $\mu X.\mathbf{0}$ are guarded.

We can also define closedness by keeping track of which variables have been binded. That is,

$$\frac{}{\text{close}_P\ \{X\}\ X} \qquad \frac{\text{close}_P\ A\ (p!\ell\langle \text{e}\rangle.P)}{\text{close}_P\ A\ P} \qquad \frac{\text{close}_P\ (\bigcup_{i\in I} A_i)\ (\sum_{i\in I} p?\ell_i(x_i).P_i)}{\text{close}_P\ A_i\ P_i \quad i \in I}$$

$$\frac{}{\text{close}_P\ \emptyset\ \mathbf{0}} \qquad \frac{\text{close}_P\ (A \cup B)\ (\text{if e then } P \text{ else } Q)}{\text{close}_P\ A\ P \quad \text{close}_P\ B\ Q} \qquad \frac{\text{close}_P\ (A\backslash\{X\})\ (\mu X.P)}{\text{close}_P\ A\ P}$$

If we can deduce that $\text{close}_P\ \emptyset\ P$, we say that P is **closed**.

Looking forward into the motivation behind this (which is that guarded and closed processes can make an arbitrary number of continuations in any branch) and giving a nice inductive structure to match with substitution, we remove unions by keeping track of at least which variables are guarded and closed. This looks like

$$\frac{}{\text{guard}'_P\ \{X\}\ X} \qquad \frac{\text{guard}'_P\ \emptyset\ (p!\ell\langle \text{e}\rangle.P)}{\text{guard}'_P\ A\ P} \qquad \frac{\text{guard}'_P\ \emptyset\ (\sum_{i\in I} p?\ell_i(x_i).P_i)}{\text{guard}'_P\ A_i\ P_i \quad i \in I} \qquad \frac{\text{guard}'_P\ B\ P}{\text{guard}'_P\ A\ P \quad A \subseteq B}$$

$$\frac{}{\text{guard}'_P\ \emptyset\ \mathbf{0}} \qquad \frac{\text{guard}'_P\ A\ (\text{if e then } P \text{ else } Q)}{\text{guard}'_P\ A\ P \quad \text{guard}'_P\ A\ Q} \qquad \frac{\text{guard}'_P\ A\ (\mu X.P)}{\text{guard}'_P\ A\ P \quad X \notin A}$$

and similarly for closure,

$$\frac{}{\text{close}'_P\ \{X\}\ X} \qquad \frac{\text{close}'_P\ A\ (p!\ell\langle \text{e}\rangle.P)}{\text{close}'_P\ A\ P} \qquad \frac{\text{close}'_P\ A\ (\sum_{i\in I} p?\ell_i(x_i).P_i)}{\text{close}'_P\ A\ P_i \quad i \in I} \qquad \frac{\text{close}'_P\ B\ P}{\text{close}'_P\ A\ P \quad A \subseteq B}$$

$$\frac{}{\text{close}'_P\ \emptyset\ \mathbf{0}} \qquad \frac{\text{close}'_P\ A\ (\text{if e then } P \text{ else } Q)}{\text{close}'_P\ A\ P \quad \text{close}'_P\ A\ Q} \qquad \frac{\text{close}'_P\ (A\backslash\{X\})\ (\mu X.P)}{\text{close}'_P\ A\ P}$$

Of course, the latter is more expressive. However, we can note that if one is $\text{close}'_P\ A\ P$, then there exists a set $B \subseteq A$ such that we can infer $\text{close}_P\ B\ P$, and similarly for guardedness.

Remark 2.1.2. As an aside, the latter construction lets us do inversion on closure in an easier fashion that matches nicely with coq. Taking the if case as an example, our original inversion on close_P would say that the continuation sets must union, but in close'_P, inversion tells us it is just a subset. This takes ties away from continuations, making the structure more independent between each continuation.

Along with closure, guardedness ensures that unfoldings are productive. This means that we have the nice property that we can make an arbitrary amount of branch choices (in any path) with a finite amount of unfoldings of the μ binder, up to the unfolding defined by taking $\mu X.P$ to $P[\mu X.P/X]$.

The relationship between the two can be expressed relatively simply. That is,

Proposition 2.1.3. *Given* $\text{guard}_P\ A\ P$ *and* $\text{close}_P\ B\ P$*, we can infer that* $\text{guard}'_P\ A\ P$ *and* $\text{close}'_P\ B\ P$*.*

Conversely, given $\text{guard}'_P\ A\ P$ *and* $\text{close}'_P\ B\ P$*, we can find sets* $A' \subseteq A$ *and* $B' \subseteq B$ *such that* $\text{guard}_P\ A'\ P$ *and* $\text{close}_P\ B'\ P$*.*

Proof. Is structural induction on P. ■

Proposition 2.1.4. *If we can infer that* $\text{guard}_P\ A\ P$ *and* $\text{close}_P\ B\ P$*, then* $A \subseteq B$*.*

Proof. We do structural induction on P. The only nontrivial case is the μ case. If $P = \mu X.P'$, by inverting on both, we can infer that $\text{guard}_P\ A\ P'$ with $X \notin A$ and $\text{close}_P B' P'$ where $B' = B$ or $B' = B \cup \{X\}$. By the inductive hypothesis, $X \notin B$, so we infer $B' = B$. It then follows that $A \subseteq B$. ■

Cruicially, it follows from this fact (and how guardedness / closures are preserved under substitution) that when we look at any unrolling of a set, it remains both guarded and closed.

Proposition 2.1.5. *If we can infer that* $\text{guard}_P\ A\ P$ *or* $\text{close}_P\ B\ P$*, then* A *and* B *are both unique.*

Proof. Follows from the fact that the constructors for both relations are injective. ■

Lemma 2.1.6 (Guardedness substitution for process variables)**.** *Let* P, Q *be processes such that* $\text{guard}'_P\ \emptyset\ P$ *and* $\text{guard}'_P\ \emptyset\ Q$*. Then,* $\text{guard}'_P\ \emptyset\ P[Q/X]$.

Proof. We do structural induction on P. The proof is immediate by case distinction on the inference of guardedness. ■

Lemma 2.1.7 (Closure substitution for process variables)**.** *Let* P, Q *be processes such that* $\text{close}'_P\ A\ P$ *and* $\text{close}'_P\ \emptyset\ Q$*. Then,* $\text{close}'_P\ A\ P[Q/X]$.

Proof. Again, structural induction on P. We note the only nontrivial case is in the μ case, but this is also immediate after a case split on whether $X \in A$ or not. ■

We can now show that processes are infinitely productive.

Lemma 2.1.8. *Given a guarded and closed process* P *and any* n*, there exists a* m *such that we can make* n *communications down any branch or reach a* $\mathbf{0}$ *or a variable with at most* m *unfoldings of* μ *and* if *binders.*

Proof. We do an induction on n. The base case is trivial. In the inductive case, we first note that by the guardedness property, every path (vieweing constructors on P as nodes) reach $\mathbf{0}$ or a $p\dagger$ for $\dagger \in \{!, ?\}$. By keeping track of this "max" number of steps one needs to take to reach any continuation and doing induction on this, we can prove the statement, noting that guardedness and closures are maintained after substitution. ■

2.1.1 Designing Processes in Coq

We want to ensure that it is easy to equate processes. Specifically, we want the following:

- Equality up to alpha conversions. That is, we want to infer for instance that $\mu X.p!\ell(x).X$ and $\mu Y.p!\ell(x).Y$ are equal
- reordering of index sets, while ensuring these sets are nonempty and finite.

We also want to make sure that there is some easily inferred decreasing argument in each object so that it is easy to reason about and do induction on it.

2.2 Local Types and Trees

Local types are defined by

$$\mathbb{T} ::= \texttt{end} \mid \bigwedge_{i \in I} p?\ell_i(S_i).\mathbb{T}_i \mid \bigvee_{i \in I} q!\ell_i(S_i).\mathbb{T}_i \mid \mu \mathbf{t}.\mathbb{T} \mid \mathbf{t}$$

Local trees are defined similarly by

$$T ::= \text{end} \mid \bigwedge_{i \in I} p?\ell_i(S_i).T_i \mid \bigvee_{i \in I} q!\ell_i(S_i).T_i$$

2.3 The Merging Operator

Session Types use a variety of different merging operators depending on their needs. A more expressive merging operator often comes with time complexity tradeoffs. We first disregard this tradeoff, outlining a set of *good* merging operators and show how they satisfy useful properties when tied to projections.

Definition 2.3.1 (The Merging Operator)**.** *The merging operator $\sqcap$ is a partial binary function over local trees. We write $T_1 \sqcap T_2 = T_3$, which can be read as "T_1 and T_2 merge into T_3". In addition, the merging operator is commutative and associative. Specifically,*

- $T_1 \sqcap T_2 = T_2 \sqcap T_1$
- $(T_1 \sqcap T_2) \sqcap T_3 = T_1 \sqcap (T_2 \sqcap T_3)$

We also assume some more properties that we expect from any merging operator,

- *(Idempotency): $T \sqcap T = T$*
- *(Application): $T_1 \sqcap T_2 = T$, $T_3 \sqcap (T_1 \sqcap T_2) = T_3 \sqcap T$.*

Of course, application is trivial if we assume that the merging operator is a function, but we place this when we want to make the merging operator a (coinductive) relation.

Notation 2.3.2.

We write $\bigsqcap_{i \in I} T_i$ to denote the partial n-ary merging function that merges all the elements in I.

Definition 2.3.3 (Mergability)**.** *Given some merging function $\sqcap$, if $T \sqcap T' = T$, then we write $T \sqsubseteq T'$ and read "T' is **mergable** into T".*

Definition 2.3.4. *We call a mergnig operator **good**, if it satisfies the following properties.*

- $(G1)$: *The output of a merging operator maintains structure. That is, if $T_1 \sqcap T_2 = T$, the top layer constructors of T_1 and T are the same.*
- $(G2)$: *The continuations merges as well. That is, if $p?\{\ell_i(S_i).T_i\}_{i \in I} \sqcap p?\{\ell'_j(S'_j).T'_j\}_{j \in J} = p?\{\ell''_k(S''_k).T''_k\}_{k \in K}$ and there exists i, j, k such that $\ell = \ell_i = \ell'_j = \ell''_k$, then $T_i \sqcap T'_j = T''_k$. Similarly for $p!$.*
- $(G3)$: *The relation $\sqsubseteq$ is maintained under constructor composition.*

Lemma 2.3.5. *If $T_i \sqsubseteq T'_i$ and $\bigsqcap_{i \in I} T_i$ is defined, then $\bigsqcap_{i \in I} T'_i$ is defined and $\bigsqcap_{i \in I} T_i \sqsubseteq \bigsqcap_{i \in I} T'_i$.*

Proof. We note that

$$\bigsqcap_{i \in I} T_i = \bigsqcap_{i \in I} (T_i \sqcap T'_i) = \bigsqcap_{i \in I} T_i \sqcap \bigsqcap_{i \in I} T'_i$$

and as the left side is defined, so is the right side. ■

We want the merging operator to be good in order to prove subject reduction, thus it is also sufficient if we can find conditions for which the properties are satisfied. Note that some literature provide examples of merging operators that don't satisfy $(G2)$ and $(G3)$, including "A Very Gentle Introduction to Multiparty Session Types". However, $(G2)$ and $(G3)$ provides compositionality to the merging operator which behaves well with our proof structure (induction on a prefix of the tree), so we utilize this here.

Proposition 2.3.6. *Given any unary (linear) context $C[X]$, if for any trees T, T', $T \sqsubseteq T'$ implies $C[T] \sqsubseteq C[T']$, then we satisfy (G3). In fact this is an if and only if condition.*

Proof. This is simply a rephrasing of the definition. ■

Proposition 2.3.7. *Coinductive full merge, coinductive plain merge, and coinductive layer merge are all good merging operators.*

Proof. TODO!!!

Note that this means any properties we prove using an arbitrary good merging operator also follow for these coinductive merging operators.

3 Theorems

3.1 The Subject Reduction Proof

We assume for the rest of this section that we are working with a good merging operator and that global trees are balanced.

Definition 3.1.1. *Given a context hole tree $\mathcal{G}$ and an accompanying set of G_i over some indexing set L with $G = \mathcal{G}[G_i]_{i \in L}$, we say that this tree is **compatible** with $p \to q$ if*

- $p, q \notin \mathcal{G}$
- *for all $i \in L$, $G_i = p \to q : \{\ell_i(S_i).G'_i\}_{i \in I}$ for some indexing set I.*

Theorem 3.1.2. *Given a well formed global type G, if $G' = G \backslash p \xrightarrow{\ell} q$ is defined, there exists a context hole tree $\mathcal{G}$ equipped with the set G_i over an indexing set L with $G = \mathcal{G}[G_i]_{i \in L}$ such that this tree is compatible with $p \to q$.*

Proof. Follows from the fact G is balanced, and that the first instance of p being involved in an interaction is of the form $p \to q$.

Lemma 3.1.3. *Given a well formed global type G, if $G' = G \backslash p \xrightarrow{\ell} q$ is defined and $G \upharpoonright_p q!\{\ell_i(S_i.)T_i\}_{i \in I}$, $G \upharpoonright_q p?\{\ell'_j(S'_j).T'_j\}_{j \in J}$, then there exists $i \in I$ and $j \in J$ such that $\ell = \ell_i = \ell'_j$ and $G' \upharpoonright_p = T_i$, $G' \upharpoonright_q = T'_j$.*

Proof. Given a well formed global type G, by Theorem 3.1.2 we can obtain a context hole tree $\mathcal{G}$ equipped with the set G_i over some indexing set L such that

$$G = \mathcal{G}[G_i]_{i \in L}$$

which is compatible with $p \to q$. We proceed our proof by an induction on $\mathcal{G}$.

In the base case, we have $\mathcal{G} = [\ ]_k$ for some k. It follows that

$$G = G_k = p \to q : \{\ell''_k(S''_k).G''_k\}_{k \in K}$$

for some indexing set K. This gives, $G' = G''_k$ with $\ell''_k = \ell$. Then, we know

$$(p \to q : \{\ell''_k(S''_k).G''_k\}_{k \in K}) \upharpoonright_p q!\{\ell_i(S_i).T_i\}_{i \in I}$$
$$(p \to q : \{\ell''_k(S''_k).G''_k\}_{k \in K}) \upharpoonright_q p?\{\ell'_i(S'_i).T'_i\}_{j \in J}$$

It follows that $G''_k \upharpoonright_p T_i$ and $G''_k \upharpoonright_q T'_j$ for $i \in I$ and $j \in J$ such that $\ell = \ell_i = \ell'_j$.

In the inductive case, we have $\mathcal{G} = s \to s' : \{\ell''_k(S''_k).\mathcal{G}_k\}_{k\in K}$. It follows that $G = s \to s' : \{\ell''_k(S''_k).G''_k\}_{k\in K}$ for some G''_k with $G''_k = \mathcal{G}_k[G_i]_{i\in L}$. By compatibility conditions, we have $p, q \notin \{s, s'\}$, so we have

$$G''_k \upharpoonright_p T^{(k)} \quad \text{with} \prod_{k\in K} T^{(k)} = q!\{\ell_i(S_i).T_i\}_{i\in I}$$
$$G''_k \upharpoonright_q T'^{(k)} \quad \text{with} \prod_{k\in K} T'^{(k)} = p?\{\ell_j(S'_j).T'_j\}_{j\in J}$$

Furthermore, $G' = G\backslash p \xrightarrow{\ell} q = s \to s' : \{\ell''_k(S''_k).(G''_k\backslash p \xrightarrow{\ell} q)\}_{k\in K}$. We now focus on the projection with p. As the merging operator is good, by condition $(G1)$,

$$T^{(k)} = q!\{\ell^{(k)}_i(S^{(k)}_i).T^{(k)}_i\}_{i\in I_k}$$

For each $k \in K$, there exists an $i_k \in I_k$ such that $\ell^{(k)}_{i_k} = \ell$, and by condition $(G2)$,

$$\prod_{k\in K} T^{(k)}_{i_k} = T_i$$

where $\ell_i = \ell$. We now note that by the inductive hypothesis, for each $k \in K$,

$$(G''_k\backslash p \xrightarrow{\ell} q) \upharpoonright_p T^{(k)}_{i_k}$$

As $G' = s \to s' : \{\ell''_k(S''_k).(G''_k\backslash p \xrightarrow{\ell} q)\}_{k\in K}$, it follows that

$$s \to s' : \{\ell''_k(S''_k).(G''_k\backslash p \xrightarrow{\ell} q)\}_{k\in K} \upharpoonright_p T_i$$

We can apply the same proof to the case for participant q, and the rest of the proof follows. ■

Lemma 3.1.4. *Given a well formed global type G, if $G' = G\backslash p \xrightarrow{\ell} q$ is defined and $G \upharpoonright_r T$ for some $r \notin \{p, q\}$, then $G' \upharpoonright_r T'$ with $T \sqsubseteq T'$.*

Proof. Given a well formed global type G, by Theorem 3.1.2 we can obtain a context hole tree $\mathcal{G}$ equipped with the set G_i over some indexing set L such that

$$G = \mathcal{G}[G_i]_{i\in L}$$

which is compatible with $p \to q$. We proceed our proof by an induction on $\mathcal{G}$.

In the base case, we have $\mathcal{G} = [\ ]_k$ for some k. It follows that

$$G = G_k = p \to q : \{\ell''_k(S''_k).G''_k\}_{k\in K}$$

for some indexing set K. This gives, $G' = G''_k$ with $\ell''_k = \ell$. It then follows that

$$G \upharpoonright_r \prod_{k\in K} T_k$$

where $G''_k \upharpoonright_r T_k$ for all $k \in K$. By the self identity on merging operators, we have $T_k \sqcap T_k = T_k$, which we can extend by application to

$$T_k \sqcap (\prod_{k\in K} T_k) = (\prod_{k\in K} T_k) = T$$

Specifically, we have $T \sqsubseteq T_k$.

In the inductive case, we have $\mathcal{G} = s \to s' : \{\ell''_k(S''_k).\mathcal{G}_k\}_{k\in K}$. It follows that $G = s \to s' : \{\ell''_k(S''_k).G''_k\}_{k\in K}$ for some G''_k with $G''_k = \mathcal{G}_k[G_i]_{i\in L}$. We have three cases to consider:

- $r = s$

We have,

$$r \to s' : \{\ell''_k(S''_k).G''_k\}_{k\in K} \upharpoonright_r s'!\{l''_k(S''_k).T''_k\}_{k\in K}$$

with $G''_k \upharpoonright_r T''_k$ for $k \in K$. By the inductive hypothesis, we know

$$(G''_k \backslash p \xrightarrow{\ell} q) \upharpoonright_r \bar{T''_k}$$

with $T''_k \sqsubseteq \bar{T''_k}$
Noting $G' = r \to s' : \{\ell''_k(S''_k).(G''_k \backslash p \xrightarrow{\ell} q)\}_{k\in K}$, we have

$$r \to s' : \{\ell''_k(S''_k).(G''_k \backslash p \xrightarrow{\ell} q)\}_{k\in K} \upharpoonright_r s'!\{l''_k(S''_k).\bar{T''_k}\}_{k\in K}$$

Therefore, it suffices to show that

$$s'!\{l''_k(S''_k).T''_k\}_{k\in K} \sqsubseteq s'!\{l''_k(S''_k).\bar{T''_k}\}_{k\in K}$$

which follows from condition $(G3)$.

- $r = s'$

Follows the same exact proof steps, where instead of sending to s' we receive from s.

- $r \notin \{s, s'\}$

We have

$$s \to s' : \{\ell''_k(S''_k).G''_k\}_{k\in K} \upharpoonright_r \bigsqcap_{k\in K} T''_k$$

where $G''_k \upharpoonright_r T''_k$ for $k \in K$. Then, by the inductive hypothesis, we have

$$(G''_k \backslash p \xrightarrow{\ell} q) \upharpoonright_r \bar{T''_k}$$

with $T''_k \sqsubseteq \bar{T''_k}$. Noting that $G' = G\backslash p \xrightarrow{\ell} q = s \to s' : \{\ell''_k(S''_k).(G''_k \backslash p \xrightarrow{\ell} q)\}_{k\in K}$, it follows from Lemma 2.3.5 that

$$\bar{T} = \bigsqcap_{k\in K} \bar{T''_k}$$

is defined with $T \sqsubseteq \bar{T}$. ■

3.2 From a Coq Implementation Perspective

Taking the merging operator and merges into operator to be simply symbols and adding hypothesis when necessary, additionally adding that that mergability implies subtypes, we get the following requirements.

1. If $\bigsqcap_{i\in I} p \dagger T_i = p \dagger f$ for $\dagger \in \{!, ?\}$ and $f\ n \downarrow (_, t)$, then $\bigsqcap_{i\in I}(T_i\ n) = t$.
2. If $\bigsqcap_{i\in I} T_i = p!f$ and $f\ n \downarrow$, then for all $i \in I$, $T_i\ n \downarrow$.
3. If $\bigsqcap_{i\in I} T_i = T$, then $T \sqsubseteq T_i$
4. end $\sqsubseteq$ end

5. for any f, g, if $f\ n \sqsubseteq g\ n$ with the same domain, then $p \dagger f \sqsubseteq p \dagger g$ for $\dagger \in \{!, ?\}$

6. If $\bigsqcap_{i \in I} T_i = T$ and $T_i \sqsubseteq T_i'$, then there exists a T' such that $\bigsqcap_{i \in I} T_i' = T'$ and $T \sqsubseteq T'$

We focus slightly on coinductive full merge. Giving an explicit definition for mergability, we can solve cases 4 and 5. We need some nice notion of merging multiple items at one (i.e., doing $\bigsqcap_{i \in I}$ at once).

4 Balanced Graphs and Trees

For the rest of this section, G is an object of some arbitrary type $\mathcal{A}$.

Definition 4.0.1. *A path γ is a finite list of natural numbers that represent transitions for G.*

Definition 4.0.2. *A transition function is a partial function $f : \mathcal{A} \to \mathbb{N} \to \mathcal{A}$ that given some G of type $\mathcal{A}$ and some $n : \mathbb{N}$, returns some $G' : \mathcal{A}$.*

For any G, given a transition function f and path γ, we then define path transition F_f as follows :

- $F_f\{G\}([\,]) = G$
- $F_f\{G\}(x :: xs) = F_f\{f(G)(x)\}(xs)$

Equivalently, it is just `foldl` $f\ G\ xs$.

Definition 4.0.3. *Given a G, a transition function f, a set of elements of $\mathcal{B}$, an element $b \in B$, and some function $g_f : \mathcal{A} \to \mathcal{P}(\mathcal{B}) \biguplus \mathbb{1}$, we say that g_f is an involvement function if*

- *if $F_f\{G\}$ is defined for some non-empty path, then $g_f(G) \subseteq \mathcal{B}$*
- *if $F_f\{G\}$ is undefined for every non-empty path, then $g_f(G) = \mathbb{1}$.*

Definition 4.0.4. *Given a G, a set of elements of $\mathcal{B}$, an element $b \in B$, and some involvement function $g_f : \mathcal{A} \to \mathcal{P}(\mathcal{B}) \biguplus \mathbb{1}$, we say that G involves b if $b \in g(G)$.*

We are now ready to define what it means for G to be balanced.

Definition 4.0.5 (Balanced G)**.** *Given some G, transition function f, and involvement function g_f, we say that G is balanced if :*

- *for all paths γ where $F_f\{G\}(\gamma) = G'$ is defined, for all γ'' and $b \in g_f(F_f\{G'\}(\gamma''))$, there exists a $k \in \mathbb{N}$ such that*
  - *for all paths γ' where $F_f\{G'\}(\gamma')$ is defined, if either $|\gamma'| = k$ or $F_f\{G'\}(\gamma') = \mathbb{1}$, there exists a γ_0' and γ_1' such that $\gamma' = \gamma_0' \cdot \gamma_1'$ and $b \in g_f(F_f\{G'\}(\gamma_0'))$.*

The definition is quite convoluted, but essentially says that for any reachable element of $\mathcal{A}$ from G, if G involves some b, then there exists some finite k such that every path of length at most k goes through an element of $\mathcal{A}$ that involves b (for a sufficient f and g_f). The notion is more clear when exemplified with graphs and trees in ??????.

Lemma 4.0.6. *For any G balanced by a corresponding f, given any path γ, if $F_f\{G\}(\gamma)$ is defined, then it is balanced.*

Proof. Follows from unfolding the definition, and also using the fact that $F_f\{F_f\{G\}(\gamma)\}(\gamma') = F_f\{G\}(\gamma \cdot \gamma')$, which is a simple property of folds.

Example 4.0.7. There exists a G such that for every path γ if $F_f\{G\}(\gamma)$ is defined it is balanced, however G itself is not.

Definition 4.0.8. *For any G and f, $\mathsf{pt}(G) = \{b : b \in F_f\{G\}(\gamma), \gamma : path\}$*

Lemma 4.0.9. *For any G and f, if*

- *for all paths γ, if $F_f\{G\}(\gamma)$ is defined, then it is balanced*
- *for all n_1, n_2 where $f(G)(n_1)$ and $f(G)(n_2)$ are defined, $\mathsf{pt}(f(G)(n_1)) = \mathsf{pt}(f(G)(n_2))$*

Then G is balanced.

4.1 Liveness

Definition 4.1.1 (Typing Contexts)**.** *We first define typing contexts, which is given by*

$$\Gamma ::= \emptyset \mid \Gamma, p : T$$

Notation 4.1.2. We write Γ_1, Γ_2 for the composition of two typing contexts when there are no participant duplications between the two.

At the coq implementation level, we will instead treat typing contexts as maps from participants to local trees. Of course this allows us to define a map on infinite participants, but we do not worry about this, as for any well-formed global type, there are only finitely many participants.

So alternatively, $\Gamma : \texttt{part} \rightharpoonup \texttt{ltt}$.

Definition 4.1.3 (Transition Labels)**.** *A transition label α is defined by*

$$\begin{aligned} \alpha ::= {} & p : q \& \ell(S) \\ \mid {} & p : q \oplus \ell(S) \\ \mid {} & (p, q)\ell \end{aligned}$$

We now define reductions on typing contexts using these labels.

Definition 4.1.4 (Typing Context Reductions)**.** *The typing context translation $\xrightarrow{\alpha}$ is defined inductively by*

$$\frac{p : q\&\{\ell_i(S_i).T_i\}_{i\in I} \xrightarrow{p:q\&\ell_k(S_k)} p : T_k}{k \in I} \qquad \frac{p : q \oplus \{\ell_i(S_i).T_i\}_{i\in I} \xrightarrow{p:q\oplus\ell_k(S_k)} p : T_k}{k \in I}$$

$$\frac{\Gamma, p : T \xrightarrow{\alpha} \Gamma', p : T}{\Gamma \xrightarrow{\alpha} \Gamma'} \qquad \frac{\Gamma_1, \Gamma_2 \xrightarrow{(p,q)\ell} \Gamma_1', \Gamma_2'}{\Gamma_1 \xrightarrow{p:q\oplus\ell(S)} \Gamma_1' \qquad \Gamma_2 \xrightarrow{q:p\&\ell(S')} \Gamma_2' \qquad S \leq S'}$$

Of course, at the implementation level when we use maps, the 3rd rule is redundant, and we can treat the latter rule as a parallel map that changes function values for participants p and q.

Notation 4.1.5. We write $\Gamma \xrightarrow{\alpha}$ if there exists a Γ' such that $\Gamma \xrightarrow{\alpha} \Gamma'$. We define a reduction for typing contexts $\Gamma \to \Gamma'$ if $\Gamma \xrightarrow{(p,q)\ell}$. As always, $\to^*$ denotes the reflexive transitive closure of $\to$.

Definition 4.1.6 (Fair and Live Paths)**.** *A* ***path*** *is a possible infinite sequence of typing contexts* $(\Gamma_n)_{n\in N}$ *where* $N = \{0, 1, 2, \dots\}$ *is a set of consecutive natural numbers such that* $\forall n \in N$ *such that* $n + 1 \in N$, $\Gamma_n \to \Gamma_{n+1}$.

A path $(\Gamma_n)_{n\in N}$ *is* ***fair*** *if* $\forall n \in N$, $\Gamma_n \xrightarrow{(p,q)\ell}$ *implies that there exists* $k \in N$ *and* ℓ' *such that* $n \leq k$ *and* $\Gamma_k \xrightarrow{(p,q)\ell'} \Gamma_{k+1}$.

A path $(\Gamma_n)_{n\in N}$ *is* ***live*** *if forall* $n \in N$,

1. $\Gamma_n \xrightarrow{p:q\oplus\ell(S)}$ *implies there exists* $k \in N$ *and* ℓ' *such that* $n \leq k$ *and* $\Gamma_k \xrightarrow{(p,q)\ell'} \Gamma_{k+1}$
2. $\Gamma_n \xrightarrow{q:p\&\ell(S)}$ *implies there exists* $k \in N$ *and* ℓ' *such that* $n \leq k$ *and* $\Gamma_k \xrightarrow{(p,q)\ell'} \Gamma_{k+1}$

Intuitively, a path is fair if any possible transaction between participants eventually happens. A path is live if a participant that can make a transaction will eventually make a transaction.

Definition 4.1.7. *A typing context* Γ *is* ***live*** *if* $\Gamma \to^* \Gamma'$ *implies that all paths starting with* Γ' *that are fair are also live.*

Definition 4.1.8 (Association)**.** *A typing context* Γ *is associated with a global type, writte* $G \sqsubseteq \Gamma$ *if* $\mathrm{pt}\{G\} \subseteq \mathrm{dom}(\Gamma)$ *and* $\forall p \in \mathrm{dom}(\Gamma)$, $G \upharpoonright p \leq \Gamma(p)$

Now follows a list of theorems that are probably needed to prove liveness for associated typing contexts.

Lemma 4.1.9 (Completeness of Association)**.** *Given an associated global tree* G *and typing context* Γ, *given* $G \sqsubseteq \Gamma$, *if* $\Gamma \xrightarrow{(p,q)\ell} \Gamma'$, *there exists a* G' *such that* $G' \sqsubseteq \Gamma'$ *and* $G \xrightarrow{\alpha} G'$.

Proof. Apparently induction on typing context reductions. TODO!!!

Theorem 4.1.10. *If* $G \sqsubseteq \Gamma$ *then* Γ *is live.*

5 Representations (loosely)

It's nice to work with some implementation where we don't require alpha conversion (or can enforce this somehow). Still having trouble trying to come up with a nice intrinsic model for this in Rocq (It kind of matters when you are trying to set up some notion of bisimularity).

5.1 On Trees

At some loose level, we work with trees that have finite branches (with possibly infinite depth).

We usually reach something that looks like one of:

```
Inductive tree1 :=
    | leaf : tree1
    | node : list (nat * tree1) → tree1

Inductive tree2 :=
    | leaf : tree2
    | node : (nat → option tree2) → tree2
```

```
Inductive tree3 :=
    | leaf : tree3
    | node {k} : (fin k → option tree3) → tree3
```

But these all have downsides we want to avoid, as we can reorder lists / might have duplicate labels in `tree1`, can have infinite branching in `tree2`, and can add a silent continuation in `tree3`.

5.2 Liveness in Rocq

We want to show that if $G \sqsubseteq \Gamma$ then Γ is live. We retrace the definitions and give a proof sketch of this at the Rocq level.

A typing context is a list of participant, local-tree pairs. To not worry about reordering and duplicates of participants, we will instead consider a map

$$\Gamma : \texttt{part} \to \texttt{option ltt}$$

While this can exhibit a context that talks about infinite participants, we don't worry too much about this, as we will associate typing contexts to regular global trees, which have only finitely many participants.

Typing contexts reduce by consuming reduction pairs. Specifically, we want $\Gamma(p)$ to match with $\Gamma(q)$, then we can consider their continuations ...

A typing context Γ is live if for any reachable context $\Gamma \to^* \Gamma'$, any path starting from Γ' that is fair is also live.

Lemma 5.2.1 (Completeness of Association)**.** *Given a global tree G and typing context Γ and that $G \sqsubseteq \Gamma$, if $\Gamma \xrightarrow{(p,q)\ell} \Gamma'$, there exists a G' such that $G' \sqsubseteq \Gamma'$ and $G \xrightarrow{\alpha} G'$.*

Proof. The idea is that $G \setminus p \xrightarrow{\ell} q\, G'$ □

Proposition 5.2.2. *Let G be a global tree and let $r \in^n G$. If $G \setminus p \xrightarrow{\ell} q\, G'$, $r \in^n G'$ or $r \in \{p, q\}$.*

Proof. Structural induction on the prefix that proves $r \in^n G$. □

The proof sketch goes as follows. Consider a global tree that is associated with a typing context. Pick any p reduction. Now we have some n such that $p \in^n G$. We do induction on n. In either case, as the context path is fair, we will eventually be able to reduce the top communication on G away. This either reduces $p \in^{n-1} G$ or $n = 0$ (base case).